

Received 30 June 2025, accepted 17 July 2025, date of publication 21 July 2025, date of current version 25 July 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3591306

RESEARCH ARTICLE

Multi-Modal Comparative Analysis on Audio Dub Detection Using Artificial Intelligence

DIVYA JENNIFER DSOUZA¹, ANISHA P. RODRIGUES¹, AND ROSHAN FERNANDES²

¹Department of Computer Science and Engineering, NMAM Institute of Technology (NMAMIT), Nitte (Deemed to be University), Nitte, India

²Department of Cyber Security, NMAM Institute of Technology (NMAMIT), Nitte (Deemed to be University), Nitte, India

Corresponding author: Anisha P. Rodrigues (anishapr@nitte.edu.in)

This work was supported by the Nitte (Deemed to be University), India.

ABSTRACT Audio dubbing, or inducing fake audio clips to a genuine video or audio file, has shown growing adverse effects in the speech domain sectors, causing concerns to individuals, organizations, and society as a whole. With the advent and rapid popularity of the artificial intelligence (AI) sector, inducing these fake clips is being made possible using generative AI that enables voice cloning and real-time voice conversion from one individual to another. Misuse of this technology would lead to a breach in one's privacy and misinterpretation, thus requiring an urgent need to address the issue of real-time detection of AI-generated audio dubs and fake audio identification and prevention. This research aims at discovering dubs in audio files using machine learning and deep learning techniques in various modes of execution, such as offline or static, batch, and online or streaming means. The audio files are segmented using libROSA, the Python package for analysis of music and audio files. The river framework works in an online mode, where the processing is done faster using dictionaries with less storage space consumption, leading to a better mode of execution for critical applications in real time. Incremental processing yields improved results in real-time audio applications by enabling low-latency responses and adaptive system behavior. The proposed hybrid deep neural network (hyDNN) gave an accuracy of 99.19% on custom data in incremental mode.

INDEX TERMS Audio dub detection, fake audio, incremental learning, feature extraction, batch processing, machine learning, audio segments, deep learning.

I. INTRODUCTION

Rapid growth in the generative artificial intelligence sector has started to pose ethical threats and concerns about various social sectors and domains, including sections of speech and audio recognition sections. Converting an individual's voice, cloning it or mimicking it is no rocket science anymore and can be achieved in less time with the help of AI technologies, even by a novice computer user [1]. The clever manipulations of the publicly available videos and audios by the intruders pose security violations like identity theft and defacement issues to the person and organizations involved in the multimedia. The concern in the area of deepfake audio and videos is a serious issue and needs to be responsibly handled by the coders and developers of the technologies, the government, and the cyber officials concerned. In this paper,

we attempt to detect deep fake audio using machine learning and deep learning methods in three different modes of execution, such as static mode, batch mode, and incremental mode. This comparison would help determine the importance of identifying fake segments in real time to minimize the loss that could be caused to a greater extent if not handled prior.

A. MOTIVATION FOR THE WORK

The risk of malicious use of biometric technology has increased [2]. Hence, research that aims to curb such practices is in great need in the present scenarios. Hence, the effort is to focus on reliable techniques and methods that can minimize the misuse of technologies with a particular focus on their strategies and comparing their performance in various modes of execution. This research also focuses on optimizing pre-processing and feature selection techniques. The main contributions of this research include: 1. Provide an analysis that compares the different modes of execution, such as

The associate editor coordinating the review of this manuscript and approving it for publication was Mehdi Sookhak¹.

static, batch, and online processing. This comparison uses a publicly available DEEP VOICE audio dataset, henceforth termed dataset-1, and a custom-made audio dataset to avoid bias. This comparison mainly would help analysts to get an insight on how deep fake audio detection performs in various modes of execution. This would benefit the analysts in making suitable and accurate decisions to evaluate the models. 2. Create a custom dataset for audio dub detection to avoid potential bias. The detailed explanation on collecting the dataset and extracting and selecting the important and relevant features and their impact on the evaluation is explained in Section III-A2. 3. The importance of varying the batch sizes in batch mode execution is detailed in Sections IV-B2 and IV-C2. 4. The importance of deep learning techniques in identifying the features and detecting the fake audio segments is explained in Section IV-E.

The remainder of the paper is organized as follows. Section II explores a review of the work done so far for the detection of fake segments in audio datasets, as well as work related to real-time processing and batch processing. A detailed survey and classification of anomaly detection techniques applied to streaming data are listed. Recent works in the field of audio dub detection and techniques used to detect the fake audios are also summarized. The state-of-the-art comparison of various audio datasets along with the models applied to detect fake parts is summarized in Table 2. Section III gives a detailed description of the methodology and processing modes used in the investigation. The process of creating a custom audio dataset, the feature extraction, and the optimization process are also explained. Section IV gives results of experimental evaluation and performance metrics for the audio dataset-1 along with custom data that are mainly used to avoid any potential bias caused by the use of publicly available datasets. Section V gives a conclusion to our work by opening future scopes.

II. LITERATURE SURVEY

The authors from [3] have provided a detailed survey of the works related to audio dub detection to assist research working around this area. The models used, approaches and techniques, issues, and prospects are neatly explained and summarized. In [4], the researchers have provided a dataset called CFAD-Chinese Dataset for Audio Detection. It considers twelve types of fake audio under transcoding and noisy conditions. The paper [2] represents the audio into features presented using Mel-spectrum, MFCCs, chromagram, and spectrogram representations. Their custom architecture provided better results with the following images: chromagram and spectrogram, whereas the VGG16 architecture gave better results with the MFCC images. The researchers in [5] combine data augmentation with hybrid feature extraction methods like Synthetic Minority Over Sampling Technique (SMOTE) and Gammatone Cepstral Coefficients (GTCC) as the front end feature extraction techniques and implement back end approaches using deep

learning models like Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU). Few machine models like Random Forest and Support Vector Machine are also used. The following combination of MFCC and GTCC with SMOTE at the front-end and LSTM at the back-end outperformed other models with 99% accuracy and a 1.6% Equal Error Rate (EER). The contributions of shimmer and jitters in the audio or voice to detect fake audio are detailed in [6]. The proposed method reduced the equal error rate from 41.29% to 35.77%. The researchers in [7] focus on types of artificially synthesized audios like replay attacks, voice conversions, and text-to-speech using a deep learning model called VGGish combined with the Convolutional Block Attention Module (CBAM) to discover spoofing attacks. Their proposed approach achieved an EER of 0.52% for physical access strikes and 0.07% for logical access strikes. The paper [8] improves fake audio detection using automatic mean opinion score prediction, which is termed as MOS-FAD. According to the researchers, this technique can improve the quality of synthetic speech. In [9], authors propose a new task termed as deepfake general audio detection to locate deepfake regions. The researchers use their custom dataset named FakeSound. Even if fake audio detection has a success rate higher with specific datasets, models tend to fail to identify fake audios when datasets are provided from different distributions [10]. Hence the researchers propose a plug-in like Sample Weight Learning (SWL) called the new stable learning-based training scheme to address the issue. This technique involves addressing distribution shift by de-correlating entire chosen attributes via learning weights from training samples. To address spoofing attacks, a novel technique based on hybrid features, which includes self-attention mechanisms, is proposed in [11]. The hybrid features that are concentrated are deep learning features and Mel-spectrogram features that are extracted in parallel paths, which will later on be max-pooled together. The ResNet model is applied to it. The performance metrics obtained are as follows: Equal Error Rate (EER) of 9.67% in the physical access and 8.94% in the deep fake task. Thus increasing overall by 74.60% and 60.05%, respectively. The authors in [12] introduce the Genuine-Focused Learning (GFL) approach aimed at detecting genuine data features using Counterfactual Reasoning Enhanced Representation (CRER) for audio reconstruction that uses the Mask AutoEncoder (MAE) framework to do the job. The content-related bottleneck (BN) attributes are retrieved with an EER of 0.25% on the ASVspoof2019 dataset. In [13] the researchers make use of customised convolutional models with pre-trained models like VGG16 and MobileNet to enhance the performance and detect the fake audios in the Deepfake audio dataset. In [22] the researchers have detailed the state-of-the-art methods of deepfake audio detection and also exposed the weakness of Deep4SNet. The authors in [2] have reviewed the effect of deep learning approaches in detecting deepfake audio for forensic investigation. The paper [27]

TABLE 1. Related works to audio dub detection.

Author & Year	Dataset	Technique	Keypoints/Limitations
Chakravarty N, et. al [5] 2023	Deepfake CROss-lingual (DECRO)	MFCC+GTCC with SMOTE at front-end and LSTM at back-end	+ Data augmentation techniques with hybrid feature extraction method - Unable to handle diverse spoofing attacks
Li K et. al [6] 2023	Audio Deep Synthesis Detection (ADD)	YIN and SWIPE	+ Reduce Shimmer and jitters in voice - Multiple layers of Feature extraction
Kanwal T et. al [7] 2024	ASVspoof 2019	VGGish + Convolutional Block Attention Module (CBAM)	+ Address replay attacks, voice conversion and text-to-speech - Model's performance degrades when noisy audios are fed
Xie Z et. al [9] 2024	FakeSound	General audio pre-trained model	+ Locates deepfake regions - Model's constraints lie in domain adaptation
Wang Z et. al [10] 2021	ASVspoof	Sample Weight Learning (SWL) module	+ Evaluates datasets from different distributions - Generalization becomes better only when the number of Random Fourier Features mapping functions is higher
Huang L, et. al [11] 2024	ASVspoof 2021	Self-attention mechanism	+ Utilize Hybrid features to obtain maximum discrimination capacity - Performance can be enhanced by adding more layers or more complex DNN when resources permit
Wang X et. al [12] 2024	ASVspoof2019	Genuine-Focused Learning (GFL) framework	+ Locates genuine audio features - Needs enhancement on detection of partially spoofed speech.
Wani TM et. al [13] 2023	Deepfake audio dataset	Custom CNN + VGG16 and MobileNet	+ Multiple deep neural network models enhance the performance. - New benchmarks for the datasets needs to be validated.
Fernandes R et. al [17] 2023	Deepfake audio + Custom dataset	ML models - SVM, kNN, AdaBoost	+ Focus on Preprocessing dataset - Usage of limited machine learning models.

has introduced a deep learning framework that outperforms machine learning models by 92% in detecting fake audio. A detailed review of the existing approaches for generating and detecting fake and partially fake audios is available in [28]. In [29] authors provide a comprehensive review of modern audio deepfake detection techniques, classifying methods into machine learning (ML) and deep learning (DL) approaches. The study analyzes detection performance across

several prominent datasets, including ASVspoof and Fake or Real (FoR), and highlights the strengths and limitations of each technique. While classical ML models like SVM and Random Forests show strong results for imitation-based deepfakes, deep learning models (e.g., CNNs, RNNs, CRNN-Spoof) offer superior scalability and feature extraction. The authors also explore emerging self-supervised learning (SSL) approaches, which aim to reduce reliance on labeled data.

TABLE 2. State-of-art comparison of various models/feature selection methods on audio dub datasets.

Article & Year	Dataset	Features Selected	Classifiers	Keypoints
Singh AK et. al[19] 2021	Custom	MFCC Cepstral	Quadratic SVM	Manual Extraction of entropy features
Rodríguez-Ortega Y et. al[18] 2020	H-Voices	Time domain waveform	Logistic Regression	Manual Extraction of entropy features
Gao Y et. al[21] 2021	ASVspoof 201, FoR,RTVCspoof	2D DCT	CNN+GRUs	Spectrum augmentation, Feature normalization
Li X et.al[20] 2021	ASVspoof 2019	Spectrogram CQT,LFCC	Res2Net	Improves generalizability to unseen spoofing attacks
Camacho S et.al[23] 2021	FoR	Auto Select via CNN	Custom CNN	Raw audio transformation and Modelling
Vaidya AO et.al[24] 2024	Face Forensics++, Celeb-DF, DFDC	Extraction of facial regions	ResNeXt, GANs, LSTM, MTCNN	Captures temporal dependencies
Ballesteros et.al[25] 2021	Dataset based on hypothesis	Handcrafted & Automatic	Deep4SNet - Custom DNN	Powerful GAN-based attacks on audio-deepfake
Lv Z et.al[1] 2022	ADD2022 Challenge	LFCC, Filterbank	ECAPA-TDNN	Use of Pretrained models
Almutairi et.al[29] 2022	ASVspoof,WaveFake Fake or Real (FoR) Custom Dataset	MFCCs, spectrogram analysis	Deep Learning: CNNs, RNNs, Transformers Hybrid Models	Review Paper that emphasizes the need for robust,scalable, and interpretable models
HyDNN	Custom	Automatic	Ensemble Deep Learning	Better Generalization with improved accuracy and robustness

Key challenges identified include noise sensitivity, limited dataset diversity, generalization issues, and the complexity of detecting imitation-based fakes. The paper emphasizes the need for robust, multilingual models and improved datasets to advance detection capabilities in real-world scenarios. In [30] the authors propose a client-side tool, integrated into a dedicated video streaming player, that leverages generative AI to combine the accessibility of multilingual dubbing with the authenticity of the original speaker's performance. This approach allows a single actor to deliver their voice in any language, preserving unique vocal traits and emotional tone. In [31], a novel approach to deepfake audio detection is introduced by a hybrid classical-quantum learning model. The authors address the limitations of traditional classical models, which often require extensive computational resources and may overlook critical audio features. To overcome these challenges, researchers propose integrating Quantum Machine Learning (QML) techniques,

leveraging principles of quantum mechanics to process data more efficiently. A summarized review of the recent literature survey in the audio dub and fake audio detection is presented in Table 1.

A. LITERATURE GAP

The literature survey details the recent works carried out on the fake audio detection. Carefully going through the works leaves us with literature gaps that can be identified and opens scopes for improvement. The following are the gaps identified from the related works: 1. High-Quality and Accurate Data Challenge: The quality and accuracy of data directly influence model performance. Existing datasets may contain inherent biases or noise, especially when they are publicly available or unstructured.

Proposed Solution: Creating custom audio data tailored to the study at hand can help eliminate bias from pre-existing datasets. This ensures that the model learns from

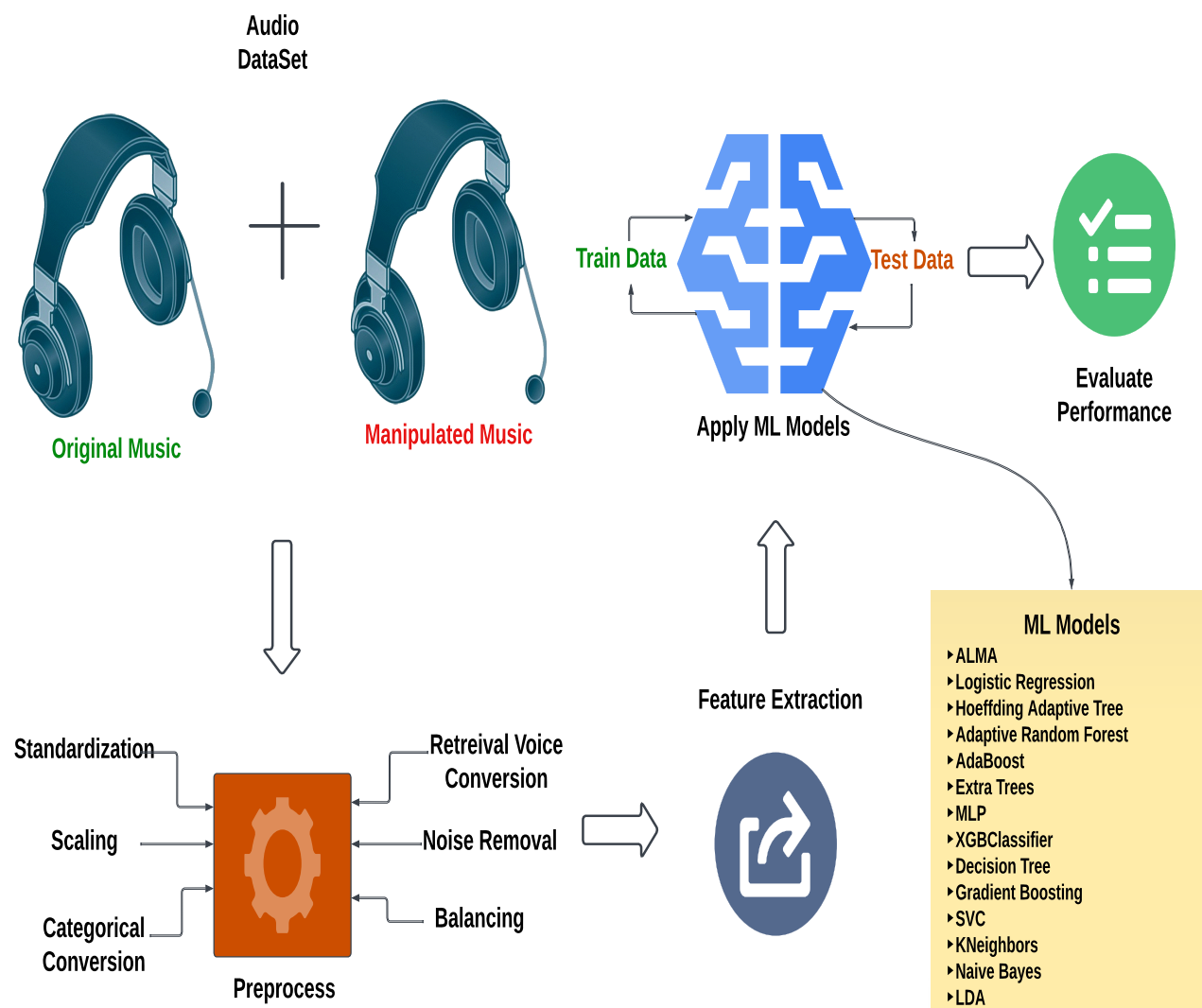


FIGURE 1. Proposed architectural diagram.

more relevant, domain-specific data, enhancing the reliability and validity of the study. Refer section III-A2 2. Feature Extraction and Selection Challenge: Extracting too many irrelevant features can overwhelm the model and reduce efficiency, causing delays without improving accuracy.

Proposed Solution: Fine-tuning feature extraction methods and carefully selecting only the most relevant features can reduce processing time and improve model performance. Using optimized parameters for feature selection ensures that computational resources are used effectively without sacrificing outcome accuracy. Refer section III-B under data preprocessing. 3. Time and Space Complexity for Real-Time Data Challenge: In real-time applications, the efficiency of a model is paramount. The model must process data quickly while using minimal computational resources to avoid delays.

High resource consumption (especially in terms of memory and processing power) can degrade performance or even cause system failures in time-sensitive tasks.

Proposed Solution: This study aims to evaluate models in three modes—static, batch, and online—to assess their resource usage and time efficiency. By comparing these modes, the research will provide insights into optimizing models for real-time data processing and resource management. Refer to Section III-E 4. Hybrid Model Challenge: Traditional audio anomaly detection methods are often constrained by limited feature representation, poor temporal modeling, and sensitivity to noise. **Proposed Solution:** Hybrid deep learning model combining Convolutional Neural Networks (CNN) and Bidirectional Long Short-Term Memory (BiLSTM) networks is used. This architecture effectively

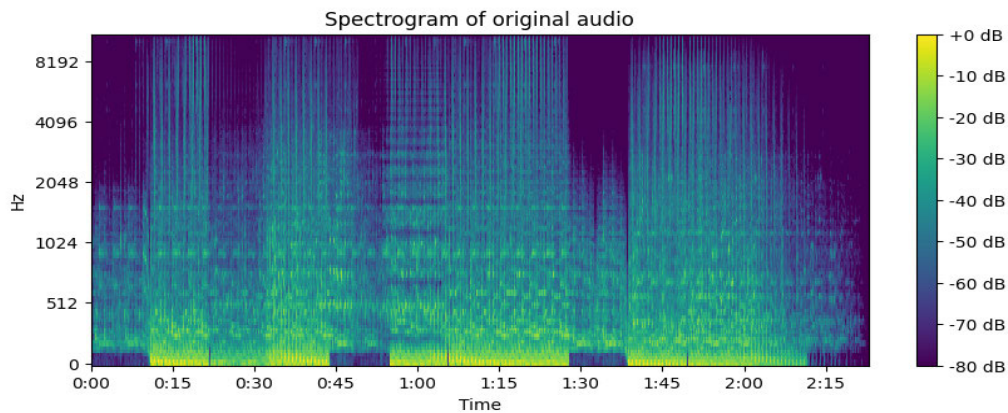


FIGURE 2. Spectrogram for the original audio.

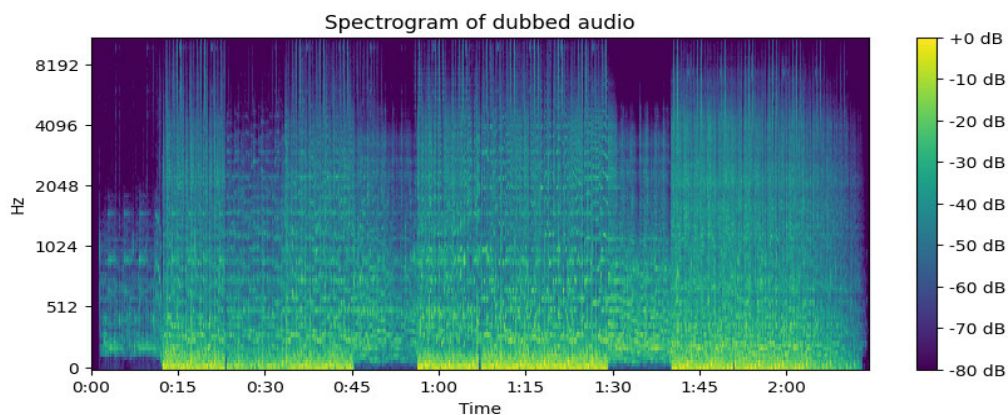


FIGURE 3. Spectrogram for the fake audio.

captures both spatial and temporal characteristics of audio signals, resulting in improved performance over traditional methods. Refer to Section IV-E.

Table 2 gives summarized details of the state-of-the-art comparison of the audio datasets, which includes the features selected for the study, classifier models considered, and the key highlights of the research.

III. METHODOLOGY

The following section describes the data collection phases, the preprocessing done on the collected samples, the experimental setup used for the research, the various modes of processing used to run the process, the relevant machine learning and deep learning models and techniques used, and the architectural flow of the process. The fig 1 shows the end-to-end process of how an audio file is processed, segmented, and evaluated to extract the features using machine learning and deep learning models and the end user being notified about the authenticity of the audio file. The steps followed are as follows: Step 1: The input audio file is fed to the pre processor. The pre processor makes sure the data is cleaned by checking for null or missing values and

substituting values wherever necessary. The noise existing in the data is eliminated using the process called retrieval voice conversion. Preprocessing also involves checking if the data is balanced, normalized, and ready to process. The audio data that contains categorical data is converted to its numerical form for seamless processing. The target values need to be balanced to train the models and get appropriate results. The data is normalized or standardized to maintain uniformity among different attribute values. Feature extraction is done to optimize the model to focus on only the priority and relevant features. Step 2: The data from the pre processing steps is fed to the model building. For the training and testing process, the data is split in the ratio of 70:30 for the model building. The machine learning and deep learning classifiers are applied to the model to build it. Custom data is also included and processed to avoid potential bias. Step 3: Three different modes of processing the audio file are initiated to compare the performance in terms of space and time complexity of various classifiers. The modes of execution include online, batch, and incremental execution. The anomaly scores and various performance evaluation metrics are compared and analyzed. The experiment is conducted

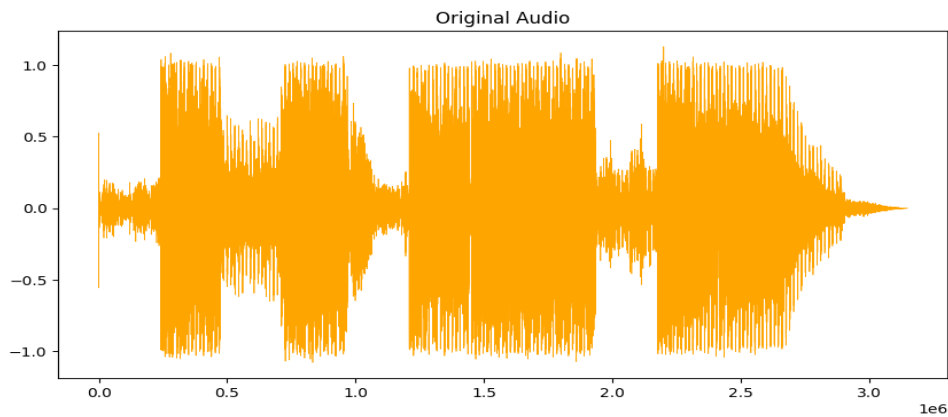


FIGURE 4. Waveform of original audio.

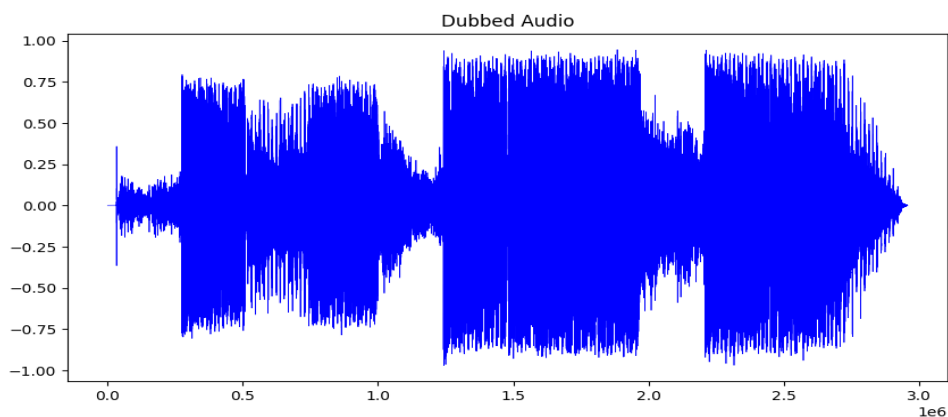


FIGURE 5. Waveform of dubbed audio.

with two different audio datasets to avoid any potential bias.

A. AUDIO DATASET COLLECTION AND PREPROCESSING

For this study, two datasets are considered. First is the Deep-Fake Voice dataset from Kaggle termed as Dataset-1. This dataset is publicly available. Second is the custom-created dataset to avoid bias that could possibly exist from the freely available dataset. The samples collected and preprocessing methods applied on both the datasets are detailed below.

1) DEEPAKE VOICE DATASET (DATASET-1)

The deepfake voice recognition dataset (Dataset-1) used for the study is taken from a publicly available domain, Kaggle, which is named as DEEP-VOICE [14]. The dataset consists of real human speech from eight popular public personalities and fake versions of those speeches achieved through retrieval-based voice conversion (RVC). The background noise was also eliminated using RVC. The original and the manipulated audio were then merged together to create the fake version of the audio. There are two forms of datasets. The first set consists of real and its manipulated version in the

audio file format, which is the .wav file format. The second form consists of the features extracted from the audio file in a one-second window segment format. The balancing is done using random sampling. There are a total of 27 features extracted, which include chromagram, root mean square, spectral centroid, spectral bandwidth, roll-off, zero crossing rate and twenty Mel-frequency cepstral coefficients with a total of 11799 samples.

2) CUSTOM DATASET

The custom-created dataset comprised songs performed by various artists as the real audios. To create the manipulated version of the songs, the cover versions of the selected songs, but performed by others, were considered and merged with original music. Each audio file was divided into 20-second segments, from which the same features as those in the original dataset were extracted. The newly created data file had 11923 rows with 27 attributes.

B. DATA PREPROCESSING

Preprocessing of the available data helps models to easily interpret the data. The process includes cleaning the

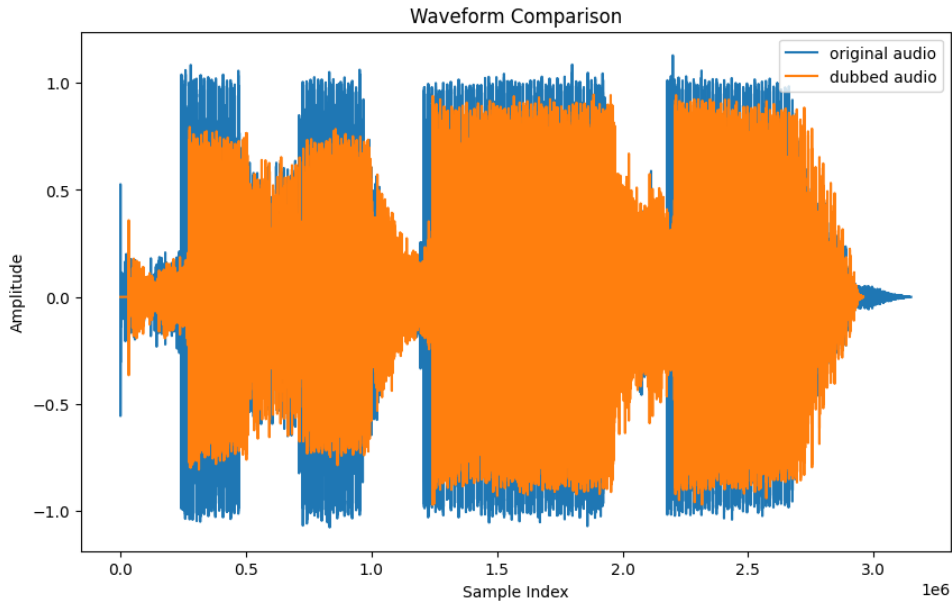


FIGURE 6. Waveform of merged audios.

TABLE 3. Dataset comparison.

Dataset	Size(samples)	Features	Modes Used	Variation
Dataset-1	11799	27	Offline	Features extracted for every one seconds of audio signal since it captures fine-grained changes in the audio signal. It is suitable for applications like speech recognition
			Batch	
			Online	
Custom Dataset	11923	27	Offline	Features extracted for every twenty second of audio signal since it summarizes features over larger chunk of time and is suitable for musical notes in long recordings.
			Batch	
			Online	

background noise, removing irrelevant attributes, filling missing values, balancing and normalizing the data, and so on. For the DeepFake data, the following preprocessing is done: Step 1: The two-stem model used in [15], known as Spleeter, is used to split the real audio file. This model has six layers of encoders followed by six layers of decoders, which works on the Convolution Neural Network’s U-Net architecture. Step 2: To manipulate the audio file to convert the audio to another individual’s voice, the retrieval-based voice conversion process is used (RVC). Step 3: Lastly, the real audio file is merged with the manipulated audio file to create the fake track. Step 4: Extract features for every one second of audio signal. A total of 26 input features and one target feature are extracted. The detailed information of the features is given in [16]. Step 5: Repeat the process for all audio files. The features are calculated using the following mathematical notations: Chromagram (C): It can be defined as the spectral information of the audio mapped to one octave. The audio was converted to chroma using

the short-time Fourier transform by combining the binning strategy. The chromagram can be defined by Short-Time Fourier Transform S of a signal at frame f and frequency bin k by the expression:

$$C(f, k) = |S(f, k)|$$

where C represents the chromagram, f represents the frame, k is the bin category, and S represents short-time Fourier transform S of a signal.

Root Mean Square (RMS): Defined as the mean value of the function, which can have both positive and negative values, like AC current and sound waves.

$$RMS(f) = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} a[n]^2}$$

where $a[n]$ is the audio signal at frame f .

Spectral Centroid (SC): Indicates the center of the mass of the spectrum, mainly used to characterize the brightness of a

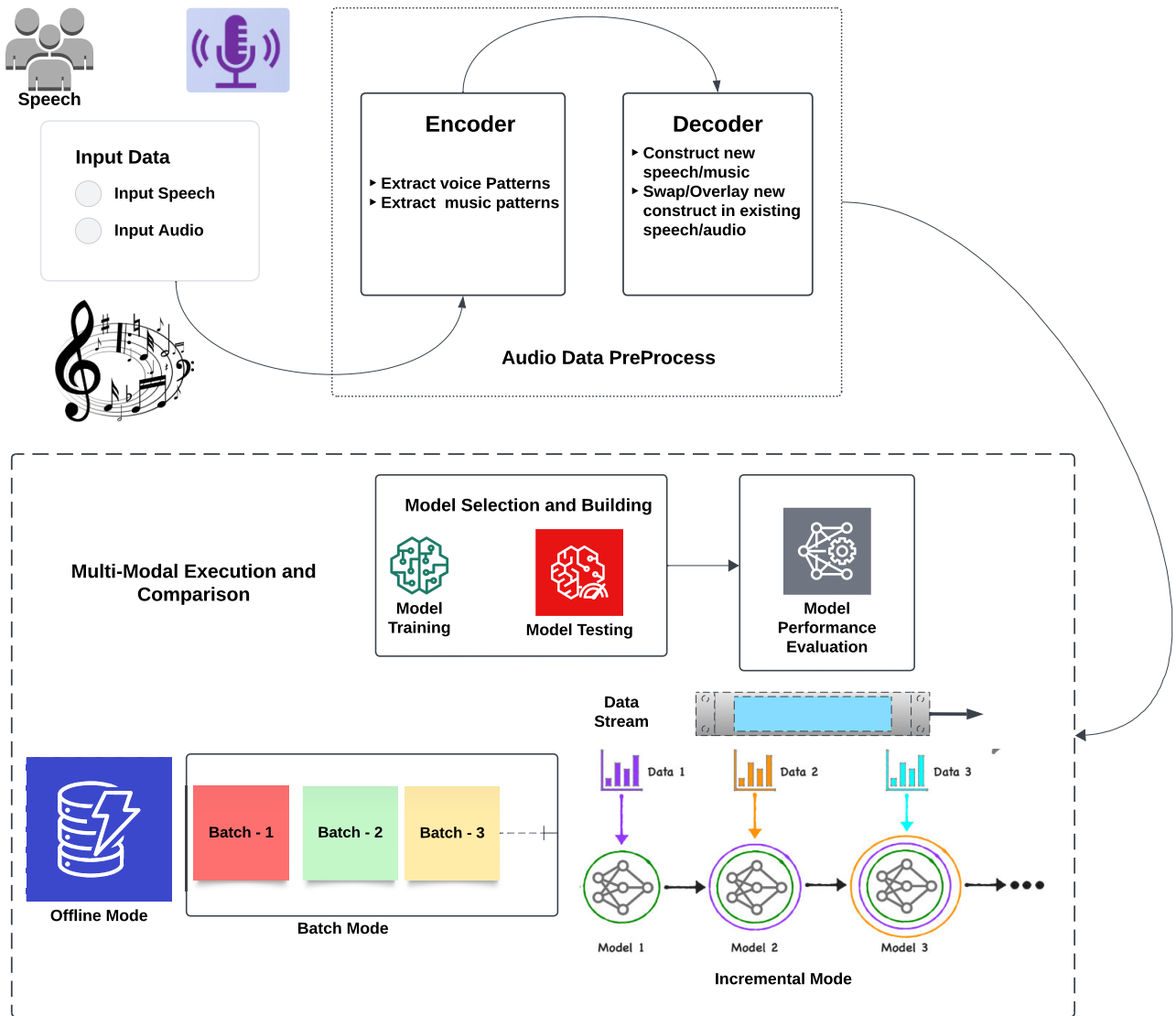


FIGURE 7. Audio processing using various modes.

sound in a spectrum.

$$SpectralCentroid(SC) = \frac{\sum_{n=0}^{N-1} c(n)a(n)}{\sum_{n=0}^{N-1} a(n)}$$

where $a(n)$ represents weighted frequency value, with bin n , and $c(n)$ represents center frequency of n .

Spectral Bandwidth (SB). It can be termed as the difference between the upper and lower frequencies of the sound. Figures 2 and 3 show the spectrogram of the original and fake audio on the custom audio dataset.

Spectral roll-off (SR): represents the frequency below which a specified percentage of the total spectral energy lies.

Zero Crossing Rate (ZCR): It can be measured as the rate at which the audio signal changes from positive to zero and then negative and vice versa. It can be obtained by dividing the number of times the signal changes by the length of the

frame.

$$ZeroCrossRate(SB) = \frac{1}{2N} \sum i |sgn(s_i) - sgn(s_{i+1})|$$

where N represents samples, sgn represents the sign function, and s_i represents the signal changing from positive to negative, crossing the value 0.

Mel-Frequency Cepstral Coefficients (MFCCs): These are a set of features used to denote the characteristics of the audio. In the first public dataset, around 20 MFCCs are taken into account, whereas for the custom dataset based on relevance, the MFCCs are reduced to 10.

Due to data being imbalanced since real audio files are larger in number than fake data, balancing at a 1:1 ratio between real and fake speech by undersampling the data belonging to the fake class is carried out.

Algorithm 1 Hybrid Deep Neural Network (HyDNN) Architecture

```

0: CNN Branch:
0:  $inputs \leftarrow \text{Input}(shape = (48, 1))$ 
0:  $x \leftarrow \text{Conv1D}(filters = 16, kernel\_size = 3, strides = 1)(inputs)$ 
0:  $x \leftarrow \text{BatchNormalization}()(x)$ 
0: for  $i = 1$  to 3 do
0:    $y \leftarrow \text{Conv1D}(filters = 32, kernel\_size = 4, strides = 1, padding = 'same')(x)$ 
0:    $y \leftarrow \text{BatchNormalization}()(y)$ 
0:    $y \leftarrow \text{Activation}("relu")(y)$ 
0:    $y \leftarrow \text{Conv1D}(filters = 16, kernel\_size = 2, strides = 1, padding = 'same')(y)$ 
0:    $y \leftarrow \text{BatchNormalization}()(y)$ 
0:    $y \leftarrow \text{Concatenate}()([y, x])$ 
0:    $x \leftarrow \text{Activation}("relu")(y)$ 
0:    $x \leftarrow \text{MaxPooling1D}(pool\_size = 3, strides = 2)(x)$ 
0: end for
0:  $cnn\_features \leftarrow \text{Flatten}()(x)$ 
0:  $cnn\_features \leftarrow \text{Dense}(30)(cnn\_features)$ 
0: LSTM Branch:
0:  $l_0 \leftarrow \text{Bidirectional}(\text{LSTM}(512, return\_sequences=True))(inputs)$ 
0:  $x \leftarrow \text{Concatenate}(axis=2)([inputs, l_0])$ 
0:  $l_1 \leftarrow \text{Bidirectional}(\text{LSTM}(256, return\_sequences=True))(x)$ 
0:  $x \leftarrow \text{Concatenate}(axis=2)([l_0, l_1])$ 
0:  $l_2 \leftarrow \text{Bidirectional}(\text{LSTM}(128, return\_sequences=True))(x)$ 
0:  $lstm\_merged \leftarrow \text{Concatenate}(axis=2)([l_0, l_1, l_2])$ 
0:  $lstm\_features \leftarrow \text{Dense}(128, activation="selu", kernel\_initializer='lecun\_normal')(lstm\_merged)$ 
0:  $lstm\_features \leftarrow \text{Dense}(1)(lstm\_features)$ 
0:  $lstm\_features \leftarrow \text{Flatten}()(lstm\_features)$ 
0: Fusion and Output:
0:  $merged \leftarrow \text{Concatenate}()([cnn\_features, lstm\_features])$ 
0:  $output \leftarrow \text{Dense}(1, activation="sigmoid")(merged)$ 
0:  $model \leftarrow \text{Model}(inputs=[inputs], outputs=output)$ 

```

For custom data, the following preprocessing is performed: Steps 1, 2, and 3 are the same as those performed for Dataset-1. Step 4: Extract features for every twenty seconds of audio signal. A total of 26 input features and one target feature are extracted. Step 5: Repeat the process for all audio files.

Standard Scaler was used to scale numerical values to have a mean of 0 and a standard deviation of 1. By putting the data in their proper perspective, this procedure helped to become more centered and more suited for different algorithms that take the normal distribution for granted. Using the Binarizer label, numerical values were additionally converted from category values. The ensemble of random forest feature importance score (RFFIS) and heat map correlation (HMC) techniques were applied to optimally choose the features of the dataset. The chosen features then underwent training and testing for model building. The ensemble technique has the advantage of being robust against overfitting, handling continuous and categorical values, effectively managing missing data, interpreting complex and multidimensional data, and many more. Figures 4, 5, and 6 show the waveforms of the original, dubbed, and merged custom audio data.

Table 3 gives the comparison of the dataset in terms of size, modalities used, variations, and complications justify the dataset considered in the investigation.

C. SAMPLING FREQUENCY AND AUDIO ALIGNMENT

The following section addresses the issues of sampling frequency and audio alignment. *Sampling frequency*: The audio data was sampled for a duration of twenty seconds per instance. To mitigate the effects of frequency aliasing, anti-aliasing techniques were employed, including low-pass filtering using Python libraries such as librosa and scipy. These filters ensure that frequency components above the Nyquist limit are attenuated, thereby preserving the integrity of frequency-based features. The analysis involved comparing the spectral characteristics of the cover song with the original recording to determine similarity. As identical frequency components are unlikely to exist in both audios unless they are essentially the same recording, this comparison serves as a robust feature for classification. The custom dataset was used for both training and validation purposes. This allowed the model to learn domain-specific characteristics effectively, while also enabling a realistic evaluation of its performance on data representative of the target application. *Audio Alignment and Speed Matching*: To address potential temporal misalignments introduced by combining cover and original songs, audio samples were carefully selected with the same tempo and duration. This ensured that the model's focus remained on distinguishing acoustic and spectral variations, particularly noise and tonal differences, rather than being influenced by timing discrepancies. Frequency remained the primary distinguishing feature for the classification task.

D. EXPERIMENTAL SETUP

Google Colab software is used to conduct the research experiment. The libraries and framework used are in the Python language. The Python packages used for audio file processing are libROSA, which is used to analyse music and audio files; pydub to simplify audio manipulation; and pyAudio for real-time processing of audio files. The system contains 16 GB of RAM, a 64-bit operating system, an x64-based processor, and the Windows 10 operating system.

E. MODES OF PROCESSING

The main focus of the study is to compare the way audio processing is achieved in various modes of execution. The modes that were experimented with in the research are as follows: i). Static mode: In static mode, the data are collected offline. This input that must be fed to the model needs to be collected in advance. The advantage of this mode is that the data can be replayed or re-processed at any point in time to the model, and results can be evaluated at different time intervals. The drawback of using this mode depends on the size and criticality of the data that needs to be processed. Huge data will lead to storage and processing issues, whereas time-critical data, such as real-time data

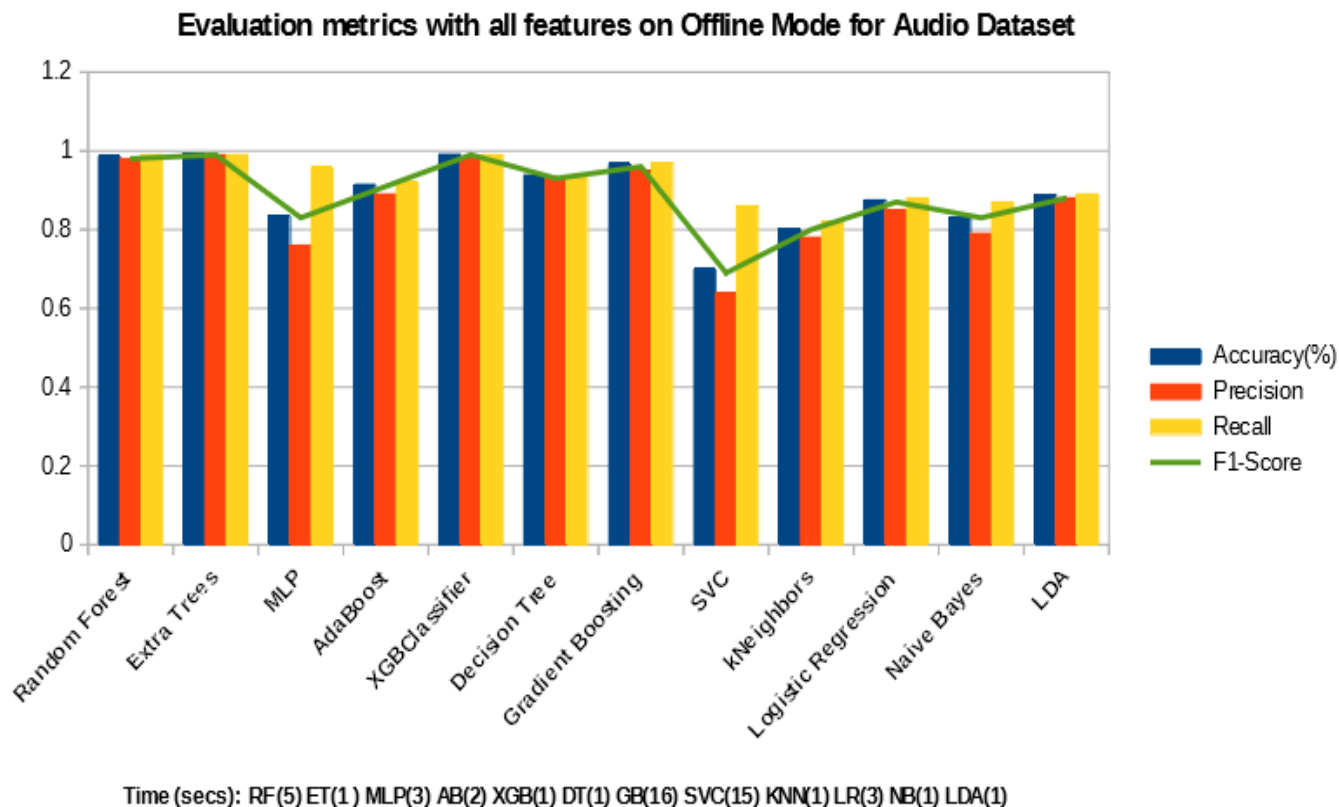


FIGURE 8. Metrics evaluation on audio dataSet-1 in offline mode with all features.

that needs to be processed on the go, cannot be processed effectively. The Python framework is used for the research with various libraries and Python packages to preprocess, such as sklearn, pandas, and numpy, to process and evaluate the machine learning and deep learning models. ii). Batch Mode: In the batch mode of execution, the input data that is to be fed is divided into chunks. For instance, for the audio input, the data are batched in certain frames. The frames are then fed into the model in batches. This type of processing is useful when one is unaware of the incoming time of the audio data. The data is fed to the model as and when the batch gets filled or at a certain stipulated time interval as set in advance. The kind of processing has the advantage of handling time series data. The storage requirements can also be handled by discarding older batches. The limitation would be the uncertainty of the arrival of data to process. The experiment is carried out with Python's batch processing frameworks, such as PyTorch, PySpark, and Python packages like Seaborn and sklearn, to visualize and evaluate the models. iii). Online Mode. The online mode takes as input the data as and when it appears. The data is processed on the go and hence is termed as incremental mode. It is most suitable for time-critical applications where the output is expected in real time. The time and storage issues are hence resolved since the data gets processed and

then discarded. This mode works on the principle that only the changes in data, like context drifts and modifications from the normal process, get stored. Hence resolving the storage issues. Instead of the whole data, a requirement for storage is needed only for the changed data. The main drawback of this mode is that the data gets to be processed only once with no option of playback processing or repeated processing. The incremental processing is carried out using Python's online processing framework, River. The data is processed in dicts, which is a JSON-like format with key-value data for faster and adaptive processing. Figure 7 details the audio processing using various modes. The input fed includes original and dubbed speeches of various personnel and music files. The audio is preprocessed using encoders and decoders. The resulting data from the preprocessed stage is fed to model training and building using machine learning and deep learning models. This step is carried out in three modes, namely static, in batches, and incremental modes, as shown. The results of the various modes are evaluated, compared, and analysed.

F. HYBRID APPROACH

A domain-specific hybrid deep learning approach for audio processing is proposed, specifically targeting the task of audio dub detection. This method integrates convolutional

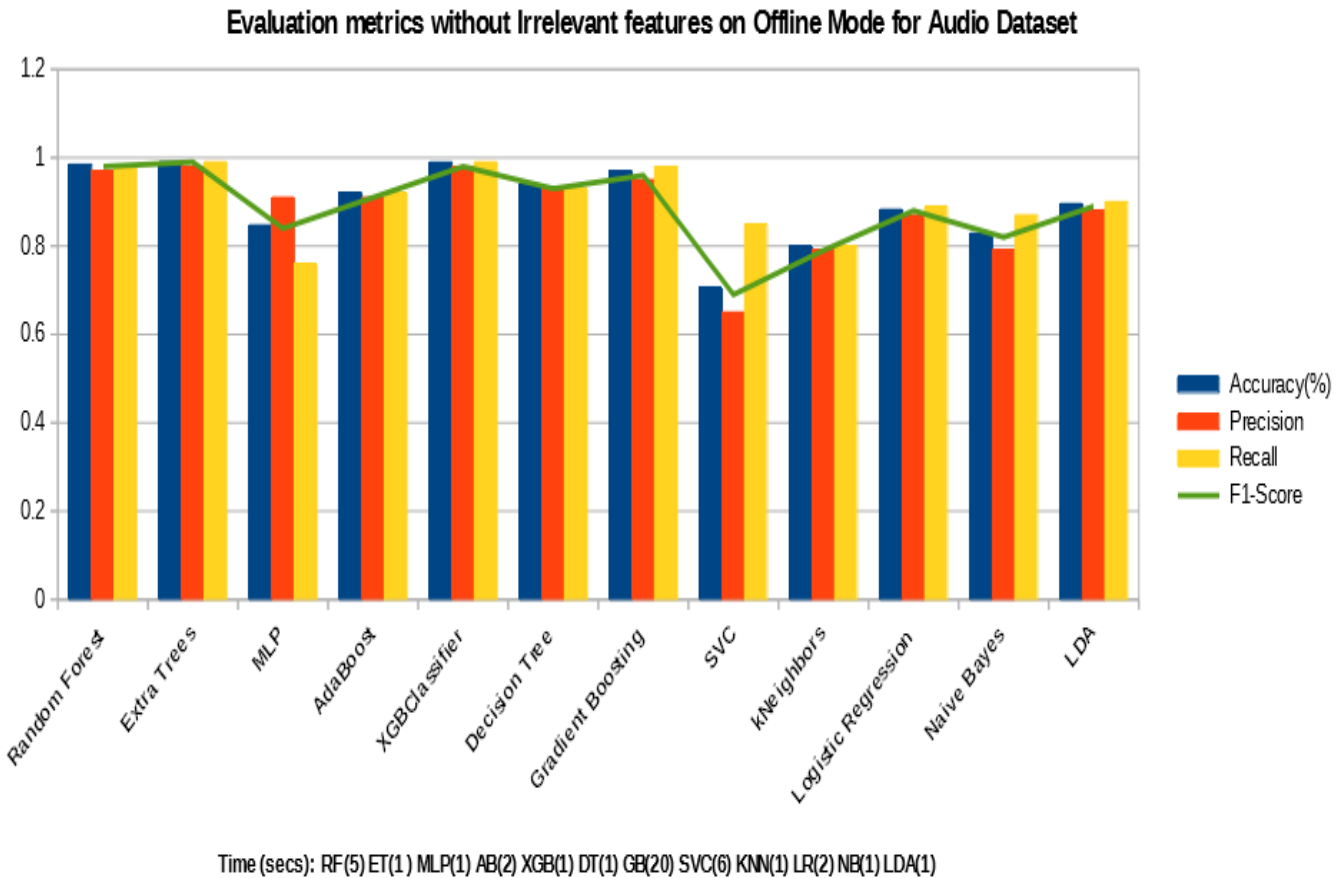


FIGURE 9. Metrics evaluation on audio dataset-1 in offline mode without irrelevant features.

TABLE 4. Evaluation metrics on batch mode (PySpark) for audio dataset-1 with all features.

Model	Accuracy(%)	time (secs)	Precision	ROC
LogisticRegression	84.67	37	0.84	0.90
Random Forest	98.46	670	0.98	0.99
GBTClassifier	95.89	1570	0.95	0.99
Decision Tree	93.37	38	0.93	0.92

TABLE 5. Evaluation metrics on batch mode (PySpark) for audio dataset-1 without irrelevant features.

Model	Accuracy(%)	time (secs)	Precision	ROC
LogisticRegression	85.21	35	0.85	0.92
Random Forest	98.44	660	0.98	0.99
GBTClassifier	96.29	1560	0.96	0.99
Decision Tree	93.55	36	0.93	0.93

neural networks (CNNs) and bidirectional long-short-term memory (BiLSTM) networks to effectively capture both

spatial and temporal patterns in audio signals. To enhance model performance and ensure domain relevance, we trained

TABLE 6. Evaluation metrics on batch mode (PyTorch) with all features for audio dataset-1 with varying batch size.

Model	Precision	Recall	F1 Score	Accuracy(%)	Time	BS
MLP	0.9953	0.9886	0.9919	99.18	1020	32
	0.9966	0.9919	0.9943	99.42	540	64
	0.9912	0.9886	0.9898	98.98	360	128
	0.9932	0.9919	0.9926	99.25	240	256
	0.9671	0.9892	0.9780	97.76	190	512
	0.9892	0.9899	0.9895	98.95	180	1024

TABLE 7. Evaluation metrics on online mode (A).

Model accuracy with all features for Audio Dataset-1		
Model	Accuracy	time (secs)
KNN	99.5	60
ALMA	70.03	5
HoeffdingTree	99.50	2
Logistic Regression	95.5	3
Perceptron	99.70	3
Hoeffding Adaptive Tree	97.05	3
Adaptive Random Forest	99.88	8
AdaBoost	97.72	16

TABLE 8. Evaluation metrics on online mode (B).

Model accuracy without extra features for Audio Dataset-1		
Model	Accuracy	time(secs)
KNN	99.5	60
ALMA	70.26	1
HoeffdingTree	99.5	3
Logistic Regression	95.4	1
Perceptron	99.71	1
HoeffdingAdaptiveTree	97.05	2
Adaptive Random Forest	99.88	8
Adaboost	97.7	13

our architecture on a custom dataset tailored to the nuances of dubbed audio characteristics. The CNN component excels at extracting local feature representations from the audio spectrograms, while the BiLSTM layers capture long-range temporal dependencies, allowing the model to distinguish subtle differences between original and dubbed audio. Experimental results demonstrate that this hybrid architecture outperforms traditional single-model baselines, yielding improved accuracy and robustness in audio dub detection tasks. A detailed pseudo code for the layering used in Hybrid Deep Neural Network (HyDNN) Architecture in explained in Algorithm 1.

IV. ANALYSIS AND RESULTS

The following section elaborates on the results obtained from processing the data in various modes along with the models and the evaluation metrics used in the process.

The audio dataset makes use of machine learning classifier models such as random forest, extra trees, Hoeffding tree, approximate large margin algorithm (ALMA), adaboost, gradient boost, K nearest neighbor, support vector, logistic regression, linear discriminant analysis (LDA), decision tree,

adaptive random forest (ARF), and naive Bayes (Bernoulli) and multinomial for the detection of dubs and fake snippets in the audio dataset. Deep learning models like multi-layer perceptron (MLP), LSTM, bidirectional LSTM, CNN, RNN, and hybrid deep learning models are also applied. The audio dub dataset has many attributes and hence needs feature extraction during data preprocessing. The custom audio dataset also undergoes feature extraction.

Some of the analysis and insights observed from processing data in various modes are as follows: 1. By processing data in offline (static) mode, the computing system would crash if the data exceeded the computing capacity. However, the temporary solution could be to use optimized data types or use sample data to save the processor; however, this would not lead to accurate results. 2. One needs to choose appropriate hardware to handle big data. However, including such hardware would lead to a cost burden. A possible solution could be to process data in chunks (mini-batches) like Dask and Spark's MLib, PySpark, or PyTorch. Another critical drawback of batch learning, or static data, is that it does not learn from new data. As and when new data arrives, the model has to learn from scratch, computing old

TABLE 9. Evaluation metrics on offline mode for custom audio dataset.

Model Accuracy and Loss with all features					
Models	Accuracy(%)	Precision	Recall	F1-Score	Time (secs)
Random Forest	97.59	0.96	0.99	0.97	5
Extra Trees	98.22	0.98	0.99	0.98	1
MLP	85.15	0.83	0.88	0.85	2
AdaBoost	90.95	0.89	0.93	0.91	3
XGBClassifier	98.45	0.98	0.98	0.98	2
Decision Tree	91.68	0.92	0.91	0.92	1
Gradient Boosting	95.96	0.95	0.97	0.96	10
SVC	70.16	0.65	0.84	0.69	5
KNeighbors	79.19	0.77	0.82	0.79	1
Logistic Regression	86.84	0.86	0.88	0.87	9
Naive Bayes	83.03	0.79	0.89	0.83	1
LDA	87.95	0.87	0.89	0.88	1

TABLE 10. Evaluation metrics on offline mode for custom audio dataset.

Model Accuracy and Loss removing irrelevant features					
Models	Accuracy(%)	Precision	Recall	F1-Score	Time (secs)
Random Forest	97.93	0.98	0.98	0.98	5
Extra Trees	98.53	0.98	0.99	0.99	1
MLP	85.57	0.9	0.80	0.86	2
AdaBoost	90.91	0.91	0.91	0.91	2
XGBClassifier	98.07	0.98	0.98	0.98	2
Decision Tree	93.32	0.94	0.93	0.93	1
Gradient Boosting	96.23	0.95	0.97	0.96	10
SVC	70.48	0.66	0.84	0.70	8
KNeighbors	78.97	0.78	0.81	0.79	1
Logistic Regression	86.59	0.86	0.87	0.87	3
Naive Bayes	82.67	0.80	0.86	0.83	1
LDA	87.85	0.87	0.88	0.88	1

and new data and creating a new dataset. This leads to annoying situations where data flows in every week, day, hour, minute, and even second in a real-time situation. 3. A feasible approach would be to process data incrementally as and when it arrives, like from the river framework [26]. It addresses the storage and computing problems of the

traditional approaches. There are also various methods in the river framework that can handle concept drift.

A. EVALUATION METRICS

The metrics were computed from the confusion matrix as follows: Accuracy = $(TP + TN) / (TP + TN + FP + FN)$

TABLE 11. Evaluation metrics on batch mode (PyTorch) with all features for custom audio dataset with varying batch size.

Model	Precision	Recall	F1 Score	Accuracy(%)	Time	BS
MLP	0.9874	0.9848	0.9861	98.59	1080	32
	0.9765	0.9867	0.9816	98.12	600	64
	0.9741	0.9921	0.9829	98.26	360	128
	0.9673	0.9756	0.9714	97.08	240	256
	0.9867	0.9835	0.9851	98.49	200	512
	0.9756	0.9802	0.9779	97.75	180	1024

TABLE 12. Evaluation metrics on batch mode (PySpark) for custom audio dataset with all features.

Model	Accuracy(%)	time (secs)	Precision	ROC
LogisticRegression	85.43	40	0.85	0.93
Random Forest	98.15	1140	0.98	0.99
GBTClassifier	95.76	1620	0.96	0.99
Decision Tree	93.02	35	0.93	0.96

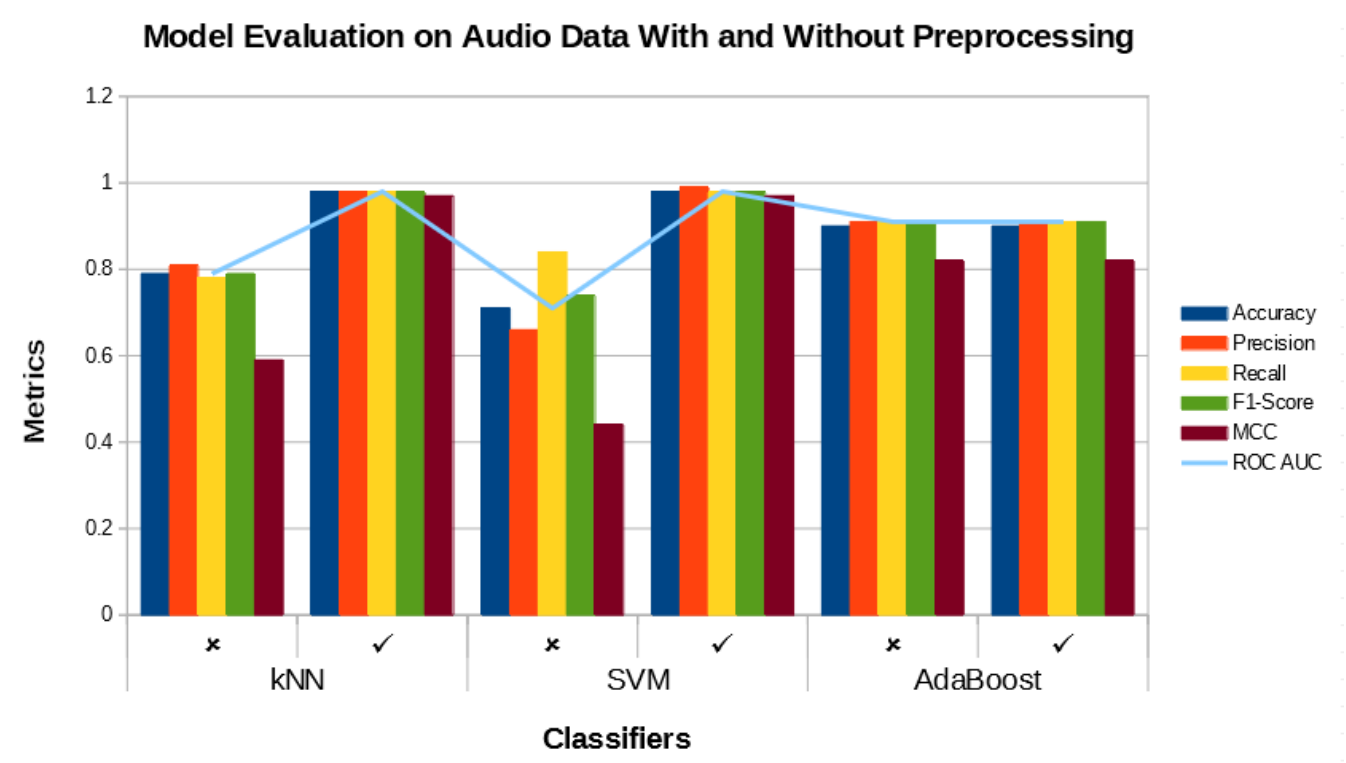


FIGURE 10. Model comparison on custom audio dataset.

Precision = TP / (TP + FP) Recall = TP / (TP + FN),
where TP = True Positive, TN = True Negative, FP = False
Positive, and FN = False Negative. F1-Score = 2 *
(Precision * Recall) / (Precision + Recall).

TABLE 13. Evaluation metrics on batch mode (PySpark) for custom audio dataset without irrelevant features.

Model	Accuracy(%)	time (secs)	Precision	ROC
LogisticRegression	84.95	36	0.85	0.92
Random Forest	97.48	660	0.97	0.99
GBTClassifier	96.26	1500	0.96	0.99
Decision Tree	93.62	43	0.94	0.94

TABLE 14. Evaluation metrics on online mode (A).

Model accuracy with all features for Custom dataset		
Model	Accuracy	time (secs)
KNN	98.72	65
ALMA	70.16	1
HoeffdingTree	98.91	3
Logistic Regression	95.24	2
Perceptron	99.35	1
Hoeffding Adaptive Tree	96.5	3
Adaptive Random Forest	99.04	8
AdaBoost	97.33	20

B. ANALYSIS AND RESULTS FOR THE AUDIO DUB DATASET-1

In the following section, the results obtained by applying machine learning and deep learning algorithms to various modes of execution on the audio dataset are shown.

1) OFFLINE MODE

The audio dub dataset is processed in offline mode with various classifiers. Offline mode refers to data that is collected beforehand. The data is then fed to the models after pre-processing. Fig. 8 and 9 show the classifiers applied to the audio dub dataset and the evaluation metrics such as accuracy, precision, recall, f1 score, and the time consumed for the different models for all features and without irrelevant features. The aim is to choose optimal attributes for the process. It can be noted that random forest, extra tree classifiers, and XGB classifiers give better results for the phishing dataset. The random forest builds multiple decision trees over subsets of data that are bootstrapped, whereas extra trees build multiple decision trees over the entire dataset. The XGB classifier is a boosting ensemble technique to classify data. The support vector classifier and the k-nearest neighbor classifier perform poorly because they do not work well with large datasets. From the graphs, one can note that there is no significant drop in accuracy even if five

attributes are removed from the dataset. Hence, eliminating unnecessary attributes can reduce computing time and storage issues.

2) BATCH MODE

In batch mode, the data is processed in chunks due to the high volume of data and the nature of the task being repetitive. Batch processing is performed using two frameworks. First is PyTorch, which is a deep learning Python library for batch processing and is ideal for research due to its flexibility and quick editing capabilities. The Pytorch model is used to run the process in various batch sizes. It can run on GPUs for faster execution. PyTorch's DataLoader module divides the training data into batches that are later processed sequentially. The second framework used is PySpark, which is the Python API. It is an open-source, distributed computing framework mainly used for real-time and large-scale data processing by processing data in parallel. The Spark session builder module assists in creating batches for processing the data. The size of the batch is specified during the coding phase. Tables 4 and 5 show the detailed overview of mini-batch processing using PySpark. Table 6 shows the evaluation metrics with all features when the multi-layer perceptron model is applied in batches of 32, 64, 128, 512, and 1024 using the Pytorch framework. The evaluation metrics, such as precision, recall, F1 score, and accuracy, are calculated. The dataset, when split into batches and fed to models, shows how accuracy varies with batch size. The time complexity to run the process is also shown. The machine learning model, the Random Forest classifier, as well as the Multi-layer Perceptron model from the deep learning approach, perform better compared to other classifiers.

3) ONLINE MODE

For the incremental mode, experiments were conducted using the River framework, which is a Python framework. All the tools in the river library can be updated with a single observation in time, making them suitable for streaming data. The experiments were conducted with real and synthetic audio datasets to understand, compare, and evaluate the space and time complexity.

Hence, the effort is to modify the existing algorithm by prioritizing the crucial features, applying the models, and comparing the outcomes. The River framework also supports

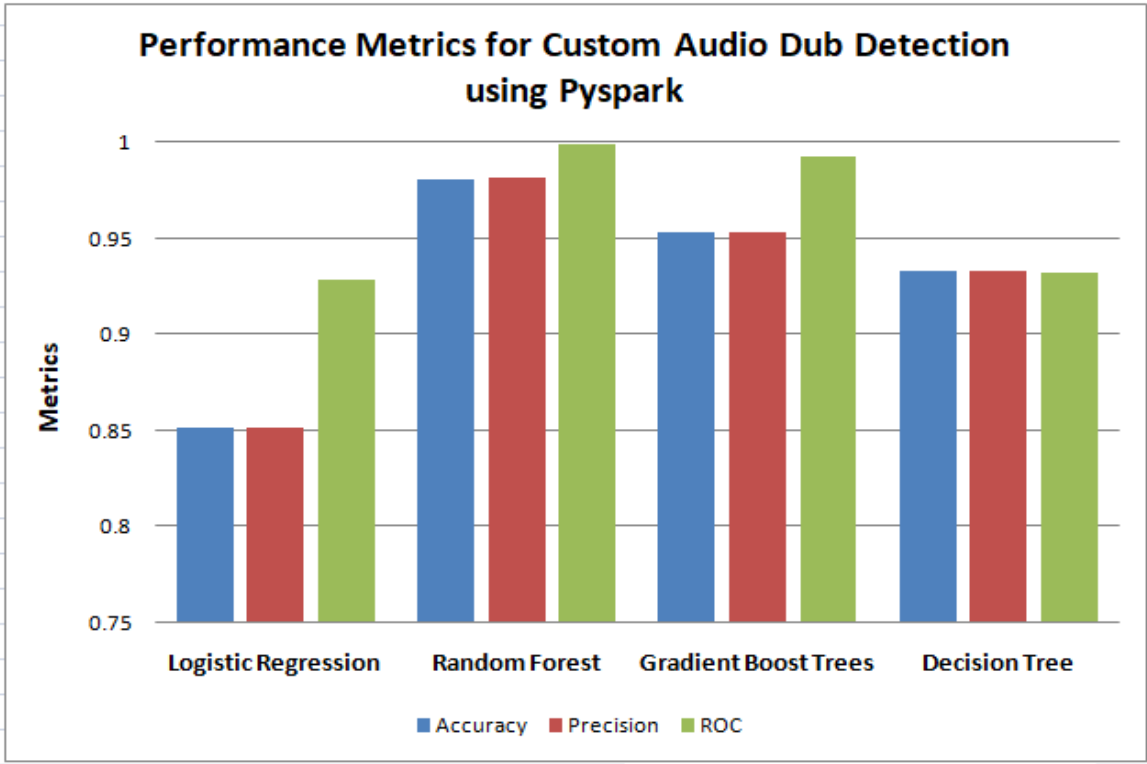


FIGURE 11. Metrics evaluation on custom audio data using PySpark.

TABLE 15. Evaluation metrics on online mode (B).

Model accuracy without extra features for Custom dataset		
Model	Accuracy	time(secs)
KNN	98.70	60
ALMA	70.35	1
HoeffdingTree	98.90	3
Logistic Regression	95.12	1
Perceptron	99.51	1
HoeffdingAdaptiveTree	96.59	3
Adaptive Random Forest	99.04	8
Adaboost	97.38	17

concept drift, which is the foremost factor in real-time applications. Tables 7 and 8 give the performance evaluation of the audio dataset - 1 on machine learning models with all features and optimized features, respectively. The adaptive random forest model and the perceptron model have better metrics compared to the other models.

C. ANALYSIS AND RESULTS FOR CUSTOM AUDIO DATASET

The following sections give an analysis of the custom data processed in various modes of execution. Custom data is

TABLE 16. Evaluation metrics using deep learning models using offline mode.

Model accuracy on optimized features for Custom dataset		
Model	Accuracy	Time(secs)
LSTM	71.87	300
CNN	41.59	150
CNN-LSTM	46.50	360
Bi-LSTM	97.09	320
CRNN	97.63	300
HyDNN	98.50	240

experimented with mainly to eliminate any kind of potential bias that could persist in publicly available datasets. The custom dataset consists of 50 audios collected from various sources. There were about 26 features in the data and one class label. The audios collected were preprocessed, and feature extraction techniques were applied to choose the optimal features. The fig 10 shows the impact of applying models to a custom audio dataset with and without applying preprocessing steps. It can be noted that preprocessing data is a crucial step in model building in order to obtain optimal results. The heat map correlation and random forest feature importance score techniques were used to filter the necessary features.

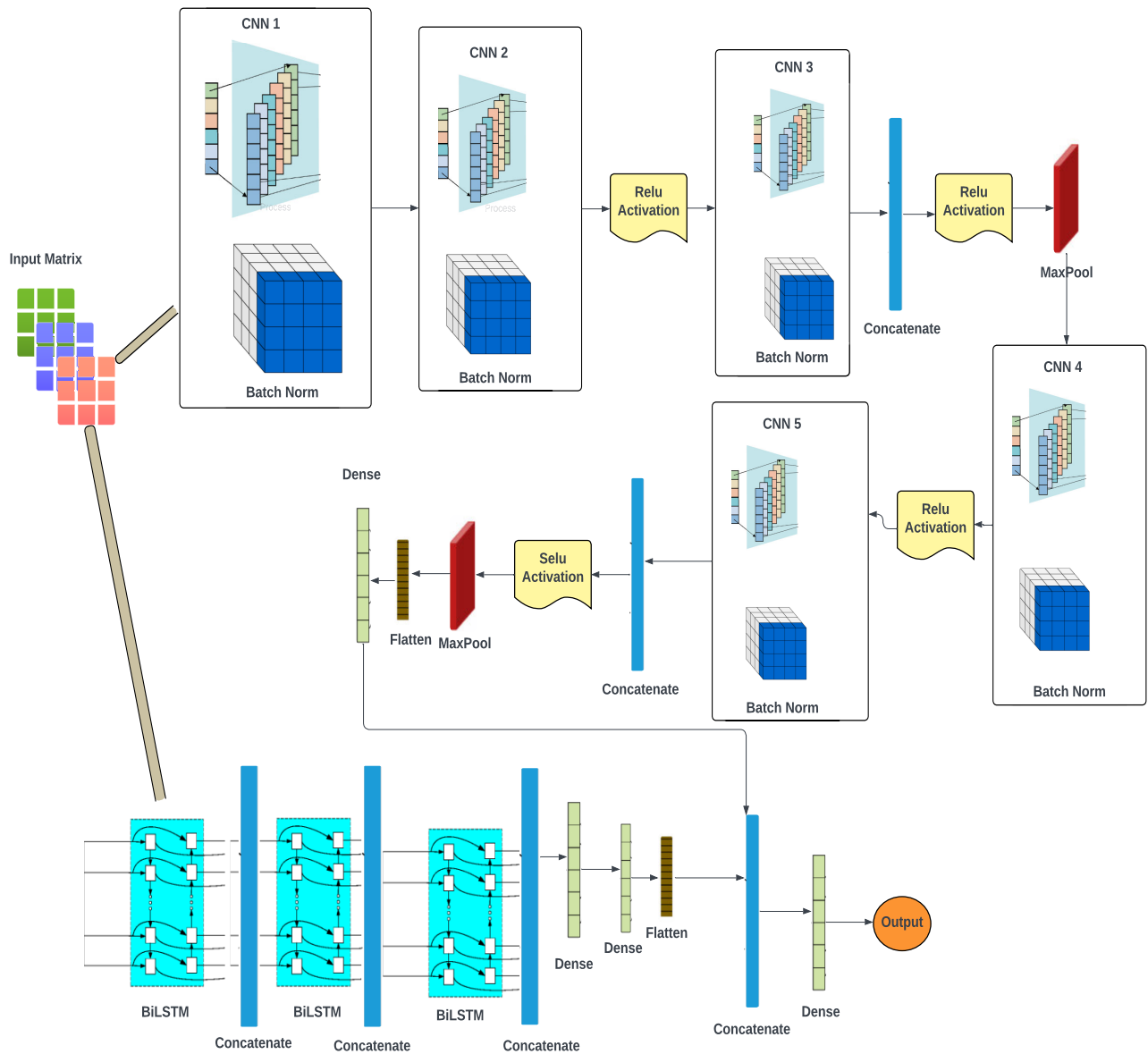


FIGURE 12. HyDNN layering architectural diagram.

After the feature extraction process, the total features were reduced to 21.

1) OFFLINE MODE

Tables 9 and 10 show the performance metrics of custom audio data on machine learning models. As seen from the results in the tables, there is a slight improvement in performance and time complexity when included with optimized features for the experiment.

2) BATCH MODE

Batch processing evaluates the data in various batches. To understand the impact of various batch sizes on the data,

an experiment was conducted with varying batch sizes. The model used was a binary neural network model with batch sizes of varying sizes, such as 32, 64, 128, 256, 512, and 1024, using the PyTorch framework. As seen in the table. 11, the analysis obtained shows that as the batch size increases, the time taken to execute decreases. The other metrics, such as accuracy, precision, recall, and the F1 score, do not vary more. The audio dataset was also executed using Pyspark on classifiers such as logistic regression, random forest, gradient boost trees (GBT) and decision trees. The random forest produced better results in terms of evaluation metrics. Tables. 12 and 13 show the results of batch processing with and without optimal features. Fig 11 depicts the performance

TABLE 17. Evaluation metrics using deep learning models using batch mode.

Model accuracy on optimized features for Custom dataset		
Model	Accuracy	Time(secs)
LSTM	98.92	117
CNN	91.12	52
CNN-LSTM	98.79	128
Bi-LSTM	98.93	147
RNN	98.24	84
CRNN	98.13	100
HyDNN	99.14	188

TABLE 18. Evaluation metrics using deep learning models using online mode.

Model accuracy on optimized features for Custom dataset		
Model	Accuracy	Time(secs)
LSTM	96.77	114
CNN	96.26	58
CNN-LSTM	98.43	154
Bi-LSTM	96.94	330
CRNN	98.30	240
HyDNN	99.19	156

metrics of the four machine learning models - logistic regression, random forest, gradient boost trees, and decision tree - using the PySpark framework.

3) INCREMENTAL MODE

The incremental mode was executed using the River Framework. The River Framework is an upcoming framework gaining popularity due to its various features in handling streaming data. It can also handle concept drift. Due to this feature, the River framework can be used for processing data with numerical attributes. For the audio dataset, the data was converted to its numerical form for the processing. Since the framework uses dictionaries to process data, like in the JSON format, computing becomes faster without requiring any additional storage. Also, in the River framework, only the modified data is stored rather than the whole data. This feature contributes to saving the storage. Tables 14 and 15 show the performance metrics on custom data with all the data and with optimized features, respectively.

TABLE 19. Evaluation metrics using deep learning models using online mode with train-on-one, test-on-another approach.

Model accuracy using DeepFake and Custom Dataset		
Model	Accuracy	Time(secs)
LSTM	92.29	360
CNN	90.56	180
CNN-LSTM	88.45	410
Bi-LSTM	97.93	400
CRNN	89.49	240
HyDNN	98.07	220

TABLE 20. Evaluation metrics using deep learning models using online mode with train-on-one, test-on-another approach.

Model accuracy using H-Voice and Custom Dataset		
Model	Accuracy	Time(secs)
LSTM	91.69	34
CNN	90.56	28
CNN-LSTM	85.46	46
Bi-LSTM	95.26	53
CRNN	88.45	29
HyDNN	97.02	52

D. EVALUATION OF MODES AND ERROR ANALYSIS

This study adopts a multi-modal comparative analysis approach in three operational settings, offline, batch, and online, to detect audio dubbing in videos. The methodology is designed to evaluate the individual strengths and limitations of audio-only systems and to demonstrate that their integration significantly enhances detection performance, particularly under complex real-world conditions.

1) AUDIO-ONLY DETECTION

Audio-only models leverage acoustic characteristics such as spectral discrepancies, prosodic patterns, and synchronization gaps to detect dubbing. Strengths: Effective in capturing unnatural speech prosody and mismatches in clean audio environments.

Limitations: Susceptible to background noise, music overlays, and poor microphone quality, which can degrade performance.

2) MODE OF OPERATION COMPARISON

The models are deployed and evaluated under the following operational scenarios:

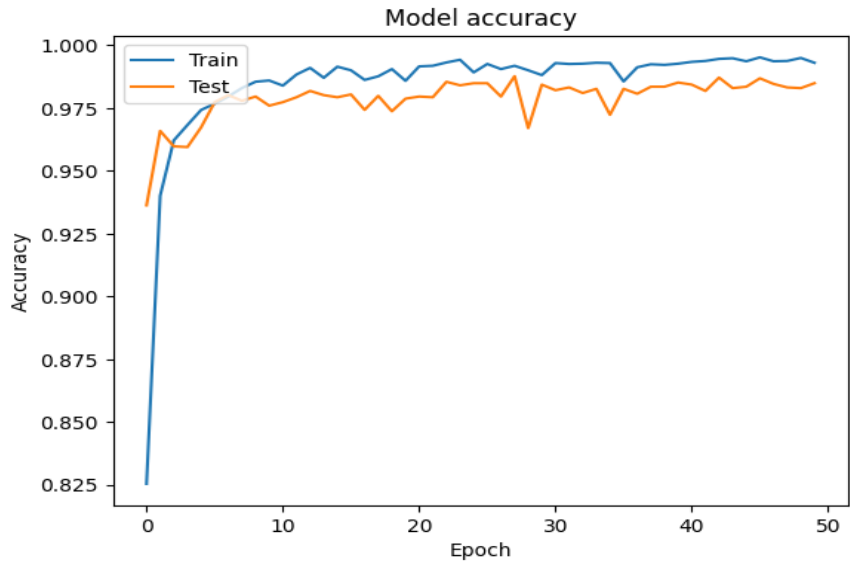


FIGURE 13. HyDNN model accuracy = 0.985%.

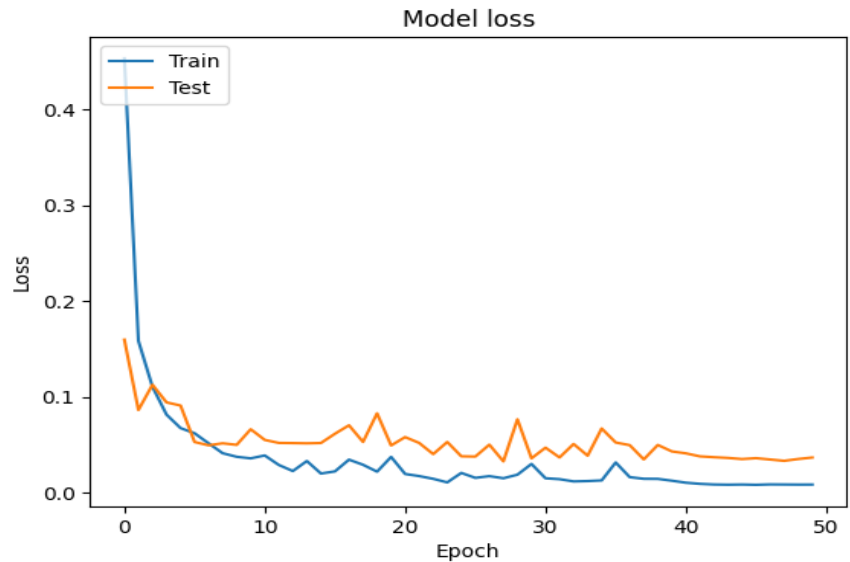


FIGURE 14. HyDNN model loss = 0.0368.

Offline Mode: The full audio context is available for processing, enabling highest detection accuracy but limited real-time utility.

Batch Mode: Processes segments sequentially, offering a balance between latency and performance.

Online Mode: Operates in real time with limited temporal context, suitable for live monitoring but with reduced accuracy.

3) BENCHMARKING UNDER REAL-WORLD CONDITIONS

To simulate real world challenges, a curated test dataset includes: songs with chorus from different artists and

background noise. Each modality is tested under these conditions to measure robustness and failure points.

E. DEEP LEARNING TECHNIQUES

Deep learning models were also used in the study to analyse the performance of the audio dataset. To show the potential overfitting issues that existed in the data, the training process was conducted to identify and plot the accuracy and loss charts on the custom audio dataset. This would provide a clearer understanding of the model's performance over time. The process was executed, including all the features, using a Keras sequential model with 50 epochs. Hyper parameters

TABLE 21. State-of-the-art methods and comparison.

Article	Dataset	Technique	Accuracy(%)
Almutairi et.al[29]	ASV spoof 2019 challenge	CNN	85.99
		SVM	71.00
Proposed		HyDNN	86.13
Almutairi et.al[29]	FoR	DNN	94.00
		Deep-Sonar	98.10
		STN	80.00
		TCN	92.0
		SVM	67
		RF	62
		KNN	62
		XGBoost	59
		LGBM	60
		CNN	88.80
Proposed		HyDNN	98.23
Almutairi et.al[29]	H-Voice	LR	98
		Deep4SNet	98.5
		Q-SVM	97.56
Proposed		HyDNN	98.66
Proposed	Custom	HyDNN	99.19

were tuned such as two layers of batch normalization with 0.3 values as dropouts were used, MaxPooling2D with pool size (2,2) from the CNN layer and number of units per LSTM layer 128 from the BiLSTM layer was set. The activations used were relu and sigmoid. The model was compiled with the Adam optimizer, and the loss metric used was binary cross-entropy. The accuracy metric used was binary accuracy. Fig12 shows the detailed layer-to-layer HyDNN architecture. The algorithm 1 details the layer-to-layer architecture to replicate the exact network with additional information. Similarly, other deep learning models used were LSTM, CNN, Bidirectional LSTM, RNN, CRNN and their ensemble models. The HyDNN model, which is the ensemble of Bi-directional LSTM and CNN, gave better performance than the other models. Figures 13 and 14 show the model accuracy and loss using the HyDNN model evaluation on the custom audio dataset with all features. The enhanced performance - an accuracy of 99.19% - can be attributed to the complementary strengths of the two architectures in the online mode. The CNN component effectively extracts spatial features from time-frequency representations of audio

signals, capturing local patterns such as harmonics, formats, and transients. These are crucial for understanding the acoustic structure. Meanwhile, the BiLSTM component models the temporal dynamics by learning both forward and backward dependencies, which is particularly beneficial in audio where context and sequence information significantly affect interpretation. The integration of these models enables the system to take advantage of both spectral and temporal features, leading to improved generalization and robustness across diverse audio inputs. Alternatively, the CRNN (Convolutional Recurrent Neural Network) model is also particularly well suited for audio dub detection due to its ability to capture both local acoustic features and temporal dependencies, two key aspects that distinguish dubbed audio from original recordings. The CRNN model was also experimented with the custom data. However, the HyDNN model gave better evaluation metrics compared to the rest of the ensemble models. Tables 16,17 and 18 show the deep learning models applied on custom data in various modes such as offline, batch and incremental modes.

The experiment was also conducted twice with the “train-on-one, test-on-another” approach, to demonstrate HyDNN’s robustness and real-world applicability. For the first experiment, the dataset used for training is DEEP VOICE audio dataset, and for testing purpose we used the custom dataset. For the second experiment, the dataset used for training is H-Voice audio dataset, and for testing purpose we used the custom dataset. The results shown in tables 19 and 20 show consistent accuracy in both datasets, with only a slight variation between training and testing performance due to inherent differences between the two data sets. The outcome shows that HyDNN maintains strong performance across datasets, thereby supporting its applicability in real-world scenarios.

V. STATE-OF-THE-ART METHODS AND COMPARISON

To assess the effectiveness of the proposed method, a detailed comparative analysis was performed against existing state-of-the-art techniques. As summarized in Table 21, the proposed approach using the incremental method consistently achieves superior performance when rerun with the same datasets using HyDNN model. This was done to ensure consistency in the evaluation. In particular, it achieves an accuracy of 99.19% in the custom dataset.

VI. CONCLUSION AND FUTURE WORKS

Audio dubbing has become a prominent cyber threat to the individuals and society as a whole. Voice cloning is done using intruders as well as with the help of various artificial intelligence technology software to create fake audios that are indistinguishable from the real audios. This study involves optimally detecting such cloned audios. From the research conducted, the following are the conclusions obtained: First, filtering out less relevant features or adding more optimal features to the available audio dataset gives us almost the same or better accuracy compared to all the features taken into consideration. This saves the overhead of pre-processing irrelevant features. Second, the experiment was conducted in offline, batch, and incremental modes for two audio datasets, mainly to check its performance and efficiency. Static and batch modes work better when the dataset is small or if the data is not very volatile. The disadvantage of batch and offline modes is the cost burden on the processing units when the data size increases. The incremental mode is a suitable solution for computing real-time critical applications because of its computing and storage advantages. As seen in the results section, the accuracy is not much compromised in the incremental learning, even though the attributes can be processed only once. One noteworthy downside in the case of incremental mode is that the features that need to be extracted for the study at one point in time may not be worthy of consideration the second time we try to compute the model or may not be available for a repeat study. One limitation of the River framework is that it works only for numerical data and not multimedia data such as images, audios, or videos. The data needs to be converted

to its numerical form before processing. Hence, processing multimedia data in an incremental setting using the River framework is still a work in progress. But there are hopeful efforts and progress in incremental frameworks, such as River, to optimize the computations and also to evaluate non-numerical data. Third, various deep learning approaches and their ensembled techniques were also experimented with on the dataset to understand their impact and performance on the audio dataset. In the machine learning techniques, adaptive random forest from the incremental framework gave better performance. This is due to the reason that in adaptive random forest the context drift data, or only the modified data is stored for future training. This approach reduces the burden of storing all data, hence giving better storage options. In the deep learning approach, Multilayer Perceptron and HyDNN gave optimal results with online mode. This shows ensembling the deep learning layers with respect to the domain will lead to better results. The enhanced performance - an accuracy of 99.19% - can be attributed to the complementary strengths of the two architectures. The CNN component effectively extracts spatial features from time-frequency representations of audio signals, capturing local patterns such as harmonics, formats, and transients. These are crucial for understanding the acoustic structure. Meanwhile, the BiLSTM component models the temporal dynamics by learning both forward and backward dependencies, which is particularly beneficial in audio where context and sequence information significantly affect interpretation. The integration of these models enables the system to leverage both spectral and temporal features, leading to improved generalization and robustness across diverse audio inputs. This multi-modal evaluation framework not only supports rigorous experimental comparison but also serves as a benchmarking paradigm for future research in dubbing detection and multimedia forensics. The comprehensive analysis substantiates the claim that combining audio and visual cues significantly enhances both accuracy and robustness in real-world deployments. As a future work, we can implement the model for visual data and also use reinforcement learning algorithms to optimize the result because the reinforcement model trains the AI model to learn sequences of actions rather than simple input-output tasks and has a huge scope for improvement.

REFERENCES

- [1] Z. Lv, S. Zhang, K. Tang, and P. Hu, “Fake audio detection based on unsupervised pretraining models,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2022, pp. 9231–9235.
- [2] M. Mcuba, A. Singh, R. A. Ikuesan, and H. Venter, “The effect of deep learning methods on deepfake audio detection for digital investigation,” *Proc. Comput. Sci.*, vol. 219, pp. 211–219, Jan. 2023.
- [3] A. Dixit, N. Kaur, and S. Kingra, “Review of audio deepfake detection techniques: Issues and prospects,” *Expert Syst.*, vol. 40, no. 8, 2023, Art. no. e13322.
- [4] H. Ma, J. Yi, C. Wang, X. Yan, J. Tao, T. Wang, S. Wang, and R. Fu, “CFAD: A Chinese dataset for fake audio detection,” *Speech Commun.*, vol. 164, Oct. 2024, Art. no. 103122.
- [5] N. Chakravarty and M. Dua, “Data augmentation and hybrid feature amalgamation to detect audio deep fake attacks,” *Phys. Scripta*, vol. 98, no. 9, Sep. 2023, Art. no. 096001.

- [6] K. Li, X. Lu, M. Akagi, and M. Unoki, "Contributions of jitter and shimmer in the voice for fake audio detection," *IEEE Access*, vol. 11, pp. 84689–84698, 2023.
- [7] T. Kanwal, R. Mahum, A. M. AlSalman, M. Sharaf, and H. Hassan, "Fake speech detection using VGGish with attention block," *EURASIP J. Audio, Speech, Music Process.*, vol. 2024, no. 1, p. 35, Jun. 2024.
- [8] W. Zhou, Z. Yang, C. Chu, S. Li, R. Dabre, Y. Zhao, and K. Tatsuya, "MOS-FAD: Improving fake audio detection via automatic mean opinion score prediction," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Apr. 2024, pp. 876–880.
- [9] Z. Xie, B. Li, X. Xu, Z. Liang, K. Yu, and M. Wu, "FakeSound: Deepfake general audio detection," 2024, *arXiv:2406.08052*.
- [10] Z. Wang, R. Fu, Z. Wen, Y. Xie, Y. Liu, X. Wang, X. Liu, Y. Li, J. Tao, Y. Lu, X. Qi, and S. Shi, "Generalized fake audio detection via deep stable learning," 2024, *arXiv:2406.03237*.
- [11] L. Huang and C.-M. Pun, "Self-attention and hybrid features for replay and deep-fake audio detection," 2024, *arXiv:2401.05614*.
- [12] X. Wang, R. Fu, Z. Wen, Z. Wang, Y. Xie, Y. Liu, J. Tao, X. Liu, Y. Li, X. Qi, Y. Lu, and S. Shi, "Genuine-focused learning using mask AutoEncoder for generalized fake audio detection," 2024, *arXiv:2406.03247*.
- [13] T. M. Wani and I. Amerini, "Deepfakes audio detection leveraging audio spectrogram and convolutional neural networks," in *Proc. Int. Conf. Image Anal. Process. Cham, Switzerland: Springer*, Sep. 5, 2023, pp. 156–167.
- [14] J. J. Bird and A. Lotfi, "Real-time detection of AI-generated speech for DeepFake voice conversion," 2023, *arXiv:2308.12734*.
- [15] R. Hennequin, A. Khelif, F. Voituret, and M. Moussallam, "Spleeter: A fast and efficient music source separation tool with pre-trained models," *J. Open Source Softw.*, vol. 5, no. 50, p. 2154, Jun. 2020.
- [16] B. McFee, C. Raffel, D. Liang, D. Ellis, M. McVicar, E. Battenberg, and O. Nieto, "Librosa: Audio and music signal analysis in Python," in *Proc. Python Sci. Conf.*, 2015, pp. 18–24.
- [17] R. Fernandes, A. D. Bhat, A. R. K. A. Diwakar, A. P. Rodrigues, D. J. Dsouza, and P. Vijaya, "Audio dub detection using machine learning," in *Proc. IEEE Int. Conf. Distrib. Comput., VLSI, Electr. Circuits Robot. (DISCOVER)*, Oct. 2024, pp. 30–34.
- [18] Y. Rodríguez-Ortega, D. M. Ballesteros, and D. Renza, "A machine learning model to detect fake voice," in *Proc. Int. Conf. Applied Inform. Cham, Switzerland: Springer*, Oct. 19, 2020, pp. 3–13.
- [19] A. K. Singh and P. Singh, "Detection of AI-synthesized speech using cepstral & bispectral statistics," in *Proc. IEEE 4th Int. Conf. Multimedia Inf. Process. Retr. (MIPR)*, Sep. 2021, pp. 412–417.
- [20] X. Li, N. Li, C. Weng, X. Liu, D. Su, D. Yu, and H. Meng, "Replay and synthetic speech detection with Res2Net architecture," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Jun. 2021, pp. 6354–6358.
- [21] Y. Gao, T. Vuong, M. Elyasi, G. Bharaj, and R. Singh, "Generalized spoofing detection inspired from audio generation artifacts," 2021, *arXiv:2104.04111*.
- [22] M. Rabhi, S. Bakiras, and R. Di Pietro, "Audio-deepfake detection: Adversarial attacks and countermeasures," *Expert Syst. Appl.*, vol. 250, Sep. 2024, Art. no. 123941.
- [23] S. Camacho, D. M. Ballesteros, and D. Renza, "Fake speech recognition using deep learning," in *Proc. Appl. Comput. Sci. Eng., 8th Workshop Eng. Appl. (WEA)*, Cham, Switzerland: Springer, 2021, pp. 38–48.
- [24] A. O. Vaidya, M. Dangore, V. K. Borate, N. Raut, Y. K. Mali, and A. Chaudhari, "Deep fake detection for preventing audio and video frauds using advanced deep learning techniques," in *Proc. IEEE Recent Adv. Intell. Comput. Syst. (RAICS)*, May 2024, pp. 1–6.
- [25] D. M. Ballesteros, Y. Rodríguez-Ortega, D. Renza, and G. Arce, "Deep4SNet: Deep learning for fake speech classification," *Expert Syst. Appl.*, vol. 184, Dec. 2021, Art. no. 115465.
- [26] D. Jennifer Dsouza, A. P. Rodrigues, and R. Fernandes, "Multi-modal comparative analysis on execution of phishing detection using artificial intelligence," *IEEE Access*, vol. 12, pp. 163016–163041, 2024.
- [27] J. Khochare, C. Joshi, B. Yenarkar, S. Suratkar, and F. Kazi, "A deep learning framework for audio deepfake detection," *Arabian J. Sci. Eng.*, vol. 47, no. 3, pp. 3447–3458, Mar. 2022.
- [28] A. Alali and G. Theodorakopoulos, "Review of existing methods for generating and detecting fake and partially fake audio," in *Proc. 10th ACM Int. Workshop Secur. Privacy Anal.*, Jun. 2024, pp. 35–36.
- [29] Z. Almutairi and H. Elgibreen, "A review of modern audio deepfake detection methods: Challenges and future directions," *Algorithms*, vol. 15, no. 5, p. 155, May 2022, doi: [10.3390/a15050155](https://doi.org/10.3390/a15050155).
- [30] E. Artioli, D. Lorenzi, F. Tashtarian, and C. Timmerer, "Generative AI for realistic voice dubbing across languages," in *Proc. 4th Mile-High Video Conf.*, Feb. 2025, pp. 75–76.
- [31] A. Pandey and B. Rudra, "Deepfake audio detection using quantum learning models," in *Proc. IEEE Middle East Conf. Commun. Netw. (MECOM)*, Nov. 2024, pp. 1–6.
- [32] S. Kairoli, B. N. Shashank, S. Kumar, A. P. Rodrigues, D. J. Dsouza, and R. Fernandes, "Video anomaly detection," in *Proc. 2nd Int. Conf. Recent Adv. Inf. Technol. Sustainable Development (ICRAIS)*, Nov. 2024, pp. 48–53.



DIVYA JENNIFER DSOUZA was born in Dubai, United Arab Emirates, in 1987. She received the B.E. and M.Tech. degrees in computer science engineering from VTU, Belagavi, India, in 2009 and 2015, respectively. She is currently pursuing the Ph.D. degree with the NMAM Institute of Technology, NITTE (Deemed to be University), India.

From 2010 to 2013, she was a Systems Engineer at Infosys Technology. From 2013 to 2022, she was an Assistant Professor at the NMAM Institute of Technology, Nitte. She is the author of 16 research articles. Her research interests include cyber security, machine learning, and deep learning.



ANISHA P. RODRIGUES was born in Mangaluru, India, in 1985. She received the B.E. degree in electrical and electronics engineering, the M.Tech. degree in computer science and engineering, and the Ph.D. degree in big data analytics from VTU, Belagavi, India, in 2006, 2012, and 2020, respectively. Since 2012, she has been an Associate Professor with the Department of Computer Science and Engineering, NMAM Institute of Technology, NITTE (Deemed to be University), India.

She is the author of three book chapters and 30 articles. Her research interests include natural language processing, machine learning, deep learning, data science, and big data analytics.



ROSHAN FERNANDES was born in Mangaluru, India, in 1979. He received the B.E. degree in computer science and engineering from Mangalore University, in 2001, and the M.Tech. degree in computer engineering and the Ph.D. degree in the area of mobile web services from VTU, Belagavi, India, in 2007 and 2019, respectively. Currently, he is a Professor and the Head of the Department of Cyber Security, NMAM Institute of Technology, NITTE (Deemed to be University), India. He has

a teaching experience of 20 years in this institution. He is the author of four book chapters and 20 articles. His research interests include mobile web services, cyber security, natural language processing, machine learning, and deep learning.

...